

Tests

Test Case Documentation

Environment for all tests:

- Python 3.13 (per user environment)
- OS: Windows 11

Module: tests/doughnut_wall_test.py — Class: TestDoughnutWall

Test: test_make_salt_length_and_randomness

- Input data:
 - Call `make_salt()` twice to produce `s1` and `s2`.
 - Verify types and lengths.
- Observed output data:
 - `s1` and `s2` are bytes-like objects.
 - `len(s1) == 16` and `len(s2) == 16`.
 - `s1 != s2` (high probability that they are different).
- Expected output data:
 - `make_salt` returns 16 cryptographically random bytes; two successive calls should be different with high probability.
- Assessment: Successful.

Test: test_hash_password_outputs_hex_and_lengths

- Input data:
 - Call `hash_password('secret-password')`.
 - Check format and lengths of returned `(hash_hex, salt_hex)`.
- Observed output data:
 - `hash_hex` and `salt_hex` contain only lowercase hex characters `[0-9a-f]`.
 - `len(hash_hex) == 128` (64 bytes dklen -> 128 hex chars).
 - `len(salt_hex) == 32` (16 bytes salt -> 32 hex chars).
- Expected output data:
 - Script-derived key of 64 bytes encoded as 128 hex chars and a 16-byte salt encoded as 32 hex chars.
- Assessment: Successful.

Test: test_hash_password_deterministic_with_same_salt

- Input data:
 - Call `hash_password('another-pass')` once to get a salt.
 - Reuse the same salt bytes to hash the same password twice.
- Observed output data:
 - The two hashes `h1` and `h2` are exactly equal.
- Expected output data:
 - With fixed password and salt, script must be deterministic and produce identical outputs.
- Assessment: Successful.

Test: test_verify_password_true_and_false

- Input data:
 - Generate `(stored_hash, stored_salt)` for password 'correct-horse-battery-staple'.
 - Verify with the correct password and with 'wrong'.
- Observed output data:
 - `verify_password` returns `True` for the correct password.
 - `verify_password` returns `False` for the wrong password.
- Expected output data:
 - `True` for matching password; `False` for non-matching password.
- Assessment: Successful.

Test: test_validate_email

- Input data:
 - Validate a list of known-valid emails and a list of known-invalid emails using `validate_email()`.
- Observed output data:
 - All valid emails returned `True`.
 - All invalid emails returned `False`.
- Expected output data:
 - Basic email format validation consistent with regex: `^[^\\s@]+@([^\\s@.]+\\.)+[\\s@.]+$`.
- Assessment: Successful.

Module: tests/db_connector_test.py — Class: TestDBConnector

Context: Each test creates a temporary SQLite database file in `setUp()` and deletes it in `tearDown()`. The schema comes from `create_tables.sql`. `DBConnector` methods under test include `user insert/fetch`, `authentication`, and `message insert/fetch with join`.

Test: test_insert_and_fetch_user

- Input data:
 - Create temp DB via DBConnector(db_name=tmp_path).
 - Call insert_user(email='alice@alice.com', name='alice', password='pw1').
 - Call fetch_user(email='alice@alice.com').
- Observed output data:
 - fetch_user returned a tuple (username, chat_color).
 - username == 'alice'.
 - chat_color matched pattern `^[0-9a-fA-F]{6}$`.
- Expected output data:
 - User insertion stores username and a random 6-hex-digit color string; fetch_user returns those values.
- Assessment: Successful.

Test: test_duplicate_user_raises_integrity_error

- Input data:
 - insert_user(email='bob@bob.bob', name='bob', password='pw2').
 - Attempt to insert the same email and username again.
- Observed output data:
 - sqlite3.IntegrityError was raised on the second insertion due to primary key (email) constraint.
- Expected output data:
 - Duplicate email insertion must violate the UNIQUE/PK constraint and raise sqlite3.IntegrityError.
- Assessment: Successful.

Test: test_authenticate_user

- Input data:
 - Call authenticate_user(email='missing', pw='x') before any insertion.
 - Insert user: insert_user(email='carol@carol.mail', name='carol', password='pw3').
 - Call authenticate_user with correct and incorrect passwords.
- Observed output data:
 - For missing user: returned (False, None).
 - For correct password: returned (True, 'carol').
 - For wrong password: returned (False, 'carol').
- Expected output data:
 - Tuple (is_valid, username) as implemented: missing user -> (False, None); correct password -> (True, username); wrong password -> (False, username).

- Assessment: Successful.

Test: test_insert_and_fetch_messages_with_join_and_filter

- Input data:
 - Insert users: ('u1@u1.u', 'u1', 'a') and ('u2@u2.u', 'u2', 'b').
 - Insert messages: ('u1@u1.u', 'hello'), ('u2@u2.u', 'world').
 - Call fetch_messages() with no last_update, then record message ids.
 - Insert another message: ('u1@u1.u', 'again').
 - Call fetch_messages(last_update=last_id_from_previous_call).
- Observed output data:
 - Initial fetch returned >= 2 rows; each row structure: (users.username, messages.*, users.chat_color).
 - The last element (users.chat_color) matched `^[0-9a-fA-F]{6}$`.
 - IDs from the messages.* portion were in increasing order.
 - With last_update filter, all returned rows had id > previous last_id, and at least one message text was 'again'.
- Expected output data:
 - fetch_messages joins users.chat_color, orders by timestamp ASC, and filters by id > last_update when provided.
- Assessment: Successful.

Manual Web Browser Testing

Prerequisites:

- Python 3.13 installed
- Project dependencies installed (see pyproject.toml)

Setup (one-time per session):

1. Start the application server:
 - Command: `python main.py`
 - Default address: <http://localhost:8080>
2. Open a modern browser (Edge, Chrome, Firefox) and navigate to <http://localhost:8080/auth>.

Manual Test Cases

1. Auth page fail
 - Input data (steps):
 - Open <http://localhost:8080/auth>

- Enter Wrong credentials rather@not.dont and password wrong; click Login.
- Observed output data:
 - The /auth page renders with title "Schätt Äbb" and Login elements.
 - /style.css returns CSS text.
 - After submitting wrong credentials, I am are redirected to /fail and see an interesting error page.
- Expected output data:
 - Login page elements are present, stylesheet served, error page served.
- Assessment:
 - Successful all elements work as described.

2. Auth page

- Input data (steps):
 - Open <http://localhost:8080/auth>
 - Enter right credentials admin and password admin; click Login.
- Observed output data:
 - The /auth page renders with title "Schätt Äbb" and Login elements.
 - /style.css returns CSS text.
 - After submitting the right credentials, I am are redirected to / and see the Chat page.
- Expected output data:
 - Login page elements are present, stylesheet served, Chat Page Served.
- Assessment:
 - Successful all elements work as described.

3. Chat page send message

- Input data (steps):
 - After login as admin, on the Chat page, enter "Hello World!" in the message input and click Send.
- Observed output data:
 - The message "Hello World!" appears in the chat history with my username and a timestamp.
 - The input field is cleared after sending.
 - My username is "admin" and my chat color is a hex color code.
- Expected output data:
 - Message appears in chat history with correct formatting.
 - Input field is cleared.
 - The username should be "admin" and the chat color should be a hex color code.
- Assessment:
 - Successful message sending and display work as described.

4. Recieve Message from another user

- Input data (steps):
 - Open a second browser or incognito window.
 - Navigate to <http://localhost:8080/auth> and sign in with username: max password: 12345
 - On the Chat page, enter "Hi Admin!" in the message input and click Send.
- Observed output data:
 - The message "Hi Admin!" appears in the chat history with the username "max" and a timestamp.
 - The input field is cleared after sending.
 - The username is "max" and the chat color is a hex color code.
- Expected output data:
 - Message appears in chat history with correct formatting.
 - Input field is cleared.
 - The username should be "max" and the chat color should be a hex color code.
- Assessment:
 - Successful message sending and display work as described.

5. Logout

- Input data (steps):
 - On the Chat page, click the Logout button.
- Observed output data:
 - I am redirected to the /auth page.
 - The session is cleared; accessing / again redirects back to /auth.
- Expected output data:
 - Logout redirects to /auth and clears the session.
 - Accessing protected routes redirects to /auth.
- Assessment:
 - Successful logout and session management work as described.